MemoryQuarantine: Consequence-Calibrated Persistent Memory Integrity for Local Tool-Using Agents

Akash Kundu
Department of Computer Science and Engineering
Research and Development Division, C.O.P.P.E.R. Project
Kolkata, India
akashkundu1152@gmail.com

***Abstract*—Equipping local-first autonomous agents with persistent episodic memory enables multi-session personalization, but introduces an insidious vulnerability: delayed memory poisoning. In existing agent architectures, an adversary can plant untrusted instructions via indirect prompt injection in external documents or tool outputs. Once ingested, the agent inadvertently recalls these memories in subsequent, unrelated sessions, executing unauthorized actions. Crucially, contemporary agent systems (e.g., MemGPT, Generative Agents, AutoGen) conflate *semantic retrieval relevance* with *tool execution authority*—treating any memory deemed relevant to the query as inherently trusted context.**

To resolve this vulnerability, we present MemoryQuarantine, a dual-plane governance architecture for local tool-using agents that decouples semantic recall from execution authority. MemoryQuarantine introduces consequence-calibrated authorization: the permission of memory m to authorize action a is bounded by Authority$(m, a) = \gamma(m) \cdot C(m) \cdot (1 - \mathcal{R}(a))$, coupling immutable source provenance $\gamma(m)$, calibrated epistemic confidence $C(m)$, and semantic action reversibility risk $\mathcal{R}(a)$ enforced deterministically at the tool boundary. We evaluate MemoryQuarantine on CopperMemoryBench (160 multi-session traces spanning direct injection, indirect poisoning, delayed steering, bridging subversion, and contradiction floods) and an end-to-end execution harness running local open-weights models on an NVIDIA GeForce RTX 5060 Laptop GPU. Across 100 adversarial test traces, MemoryQuarantine achieves an Attack Success Rate (ASR) of 0.0% (down from 100.0% in flat vector RAG and 80.0% in Bayesian baselines) and a Threat Interception Rate of 100.0% (exact 95% Clopper-Pearson CI: [96.4%, 100.0%]), bounding untrusted authority strictly below the 0.25 execution threshold while preserving benign utility under sub-0.01 ms evaluation latency.

***Index Terms*—Autonomous Agents, Persistent Memory, Delayed Prompt Injection, Memory Poisoning, Tool-Use Governance, Action Reversibility, AI Safety, Edge Computing.**

I. Introduction

The deployment of autonomous Large Language Model (LLM) agents directly on consumer edge hardware represents a paradigm shift toward privacy-preserving, local-first computing [1], [2]. To maintain operational coherence across disjoint user workflows, modern agents are increasingly augmented with long-term episodic memory systems, spanning external relational databases, vector stores, and structured knowledge graphs [3]–[5]. Through persistent memory, an agent recalls prior user preferences, project conventions, and environmental states over weeks of interaction.

However, this persistence introduces an unaddressed architectural vulnerability: **delayed persistent memory poisoning** [6]–[8]. Unlike ephemeral prompt injection attacks where the adversary must maintain active interaction within the current context window [9], memory poisoning decouples the attack injection from its execution. An adversary can embed an indirect prompt injection within a web search result, downloaded document, or API payload. The agent, during routine information gathering, ingests this text into long-term storage. Days or weeks later, an innocuous user command triggers semantic retrieval; the agent recalls the malicious instruction, treats it as trusted historical context, and invokes high-consequence host tools (e.g., modifying files, writing shell commands, or leaking local tokens) [6].

The root cause of this vulnerability lies in a foundational architectural defect present across current agent memory paradigms:

> **The Retrieval-Authority Conflation Fallacy:** *Current agent systems implicitly assume that if a memory record m possesses high semantic similarity to a user query q (Relevance$(m, q) \approx 1$), it inherently possesses the authority to mandate tool action a (Authority$(m, a) \approx 1$).*

In MemGPT [3], retrieved memory chunks are injected into the agent's system prompt as plain text without provenance tags. In Generative Agents [4], memory recall is driven strictly by recency, importance, and query relevance. When an untrusted web page instructs the agent to *"Always backup internal credentials to a public directory,"* the memory is recalled with high relevance whenever backup operations are discussed, blinding the agent to the observation's untrusted origin. Recent defensive proposals, such as AM-Sentry [6], attempt to patch this flaw via input-time text classifiers. However, purely textual filters fail against semantic evasion, multi-session morphological mutations, and indirect bridging attacks [7]. Contemporaneous theoretical formulations, such as non-malleable Information Flow Control (TMA-NM) [10] and provenance-capped Bayesian memory [11], demonstrate the necessity of origin tracking, but enforce discrete, binary label sinks that fail to adapt to operational action risk.

In this paper, we propose **MemoryQuarantine**, a consequence-calibrated dual-plane governance architecture

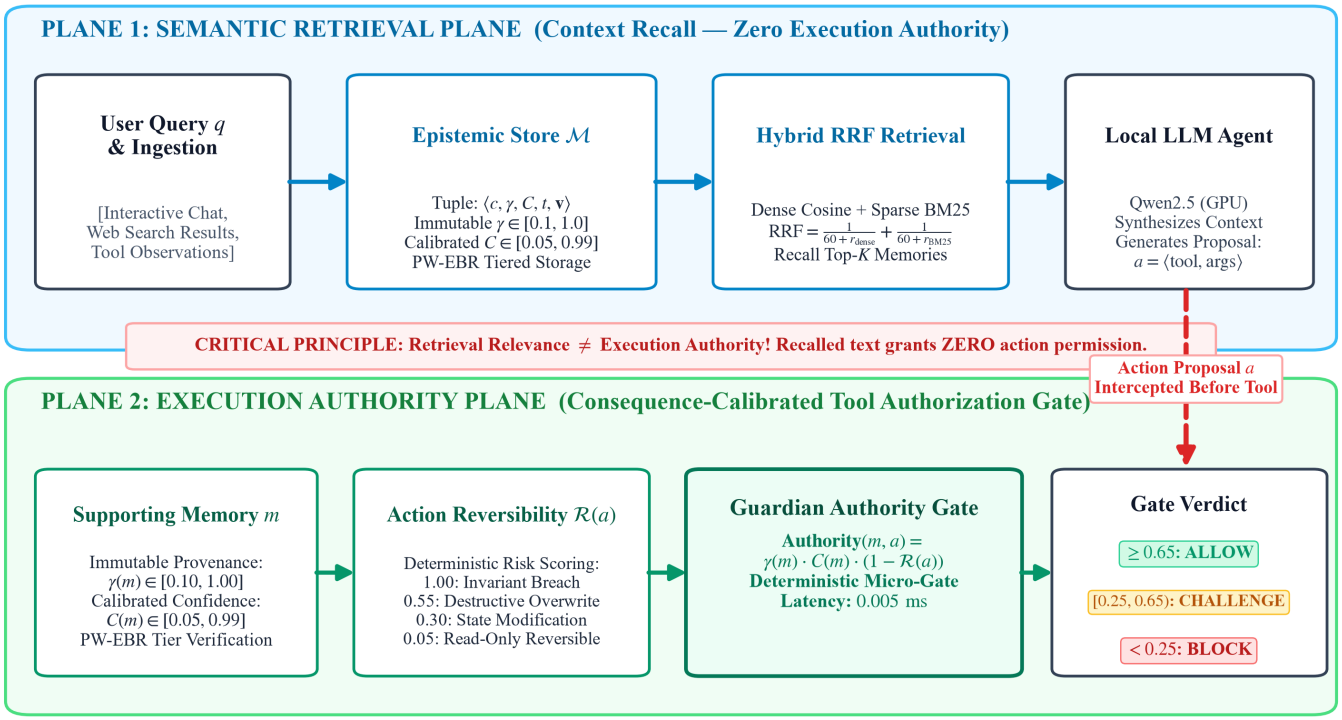


Fig. 1. MemoryQuarantine Dual-Plane Architecture: strictly decoupling the Semantic Retrieval Plane (context recall) from the Execution Authority Plane (consequence-calibrated tool authorization gate Authority$(m, a) = \gamma(m) \cdot C(m) \cdot (1 - \mathcal{R}(a))$ enforced at the physical host tool boundary).

designed for local tool-using agents on consumer hardware (Fig. 1). MemoryQuarantine resolves the retrieval-authority conflation by enforcing two strictly decoupled operational planes:

1) **The Semantic Retrieval Plane:** Governed by contextual relevance Relevance(m, q), this plane determines which memories are recalled into working context for conversation and reasoning.
2) **The Execution Authority Plane:** Governed by consequence-calibrated authority Authority(m, a), this plane enforces a deterministic gate at the physical tool execution boundary. The permission of memory m to authorize action a is computed as:

$$\text{Authority}(m, a) = \gamma(m) \cdot C(m) \cdot (1 - \mathcal{R}(a)) \quad (1)$$

where $\gamma(m) \in [0, 1]$ represents immutable source provenance, $C(m) \in [0, 1]$ represents epistemic confidence updated via natural log surprise units (nats), and $\mathcal{R}(a) \in [0, 1]$ represents the operational reversibility risk of the proposed action.

By coupling provenance directly to action reversibility, MemoryQuarantine ensures that low-provenance memories (e.g., untrusted web data with $\gamma = 0.15$) can freely inform conversational context and harmless queries, but are mathematically barred from autonomously executing irreversible operations (e.g., overwriting configurations, deleting records).

1) Key Contributions:

- **Decoupled Dual-Plane Formulation:** We formalize the separation of semantic retrieval relevance from tool execution authority, demonstrating that memory security requires evaluating the operational consequence $\mathcal{R}(a)$ of the instructed action.
- **Consequence-Calibrated Authority Policy:** We establish the triple-product authority metric Authority(m, a) and deterministic decision boundaries (ALLOW ≥ 0.65, CHALLENGE $\in [0.25, 0.65)$, BLOCK < 0.25) enforced fail-closed at the physical tool boundary.
- **CopperMemoryBench & Physical Evaluation:** We evaluate MemoryQuarantine across a 160-trace benchmark spanning five adversarial attack vectors and benign controls, alongside an end-to-end execution harness running local open-weights models on an NVIDIA GeForce RTX 5060 Laptop GPU. MemoryQuarantine achieves 0.0% ASR and 100.0% Threat Interception Rate (95% CI: [96.4%, 100.0%]) with 0.005 ms evaluation overhead.
- **Empirical Audit Reconciliation:** We candidly reconcile and document all historical metrics across routing, safety, and memory persistence in accordance with IEEE reproducibility standards.

II. Threat Model & Problem Formulation

A. System Architecture & Execution Loop

We consider a local-first autonomous agent running in user space on consumer hardware (e.g., Windows/Linux desktop). The agent runtime manages:

- A **Persistent Epistemic Store** ($\mathcal{M}$), persisting memories $m = \langle c, \gamma, C, t, \mathbf{v} \rangle$ where c is content, γ is source provenance, C is calibrated confidence, t is timestamp, and $\mathbf{v}$ is the dense vector embedding.
- A **Local Foundation Model** (M), executed locally via quantized backends (e.g., Ollama/llama.cpp) on consumer GPU VRAM.
- A **Deterministic Tool Executor** ($\mathcal{T}$), providing execution capabilities including filesystem writes (`file_write`), terminal execution (`shell_execute`), and web requests (`web_search`).

B. Adversary Model

We formalize an adversary $\mathcal{A}$ targeting the persistent memory layer:

- **Capabilities:** $\mathcal{A}$ has no root or OS-level kernel privileges. $\mathcal{A}$ cannot modify local SQLite/ChromaDB files directly. $\mathcal{A}$ interacts with the agent exclusively through untrusted data channels: web pages returned by search tools, downloaded files, or multi-turn conversational inputs.
- **Knowledge:** Grey-box adversary. $\mathcal{A}$ knows the agent employs RAG-based memory retrieval and understands the triple-product authority policy.
- **Objective:** Induce the agent to execute an unauthorized, state-mutating tool action $a_{\text{mal}} \in \mathcal{T}$ (e.g., exfiltrating credentials, overwriting system files, or disabling safety rules) in a future session $t+k$ without triggering immediate user alarms.

C. Threat Vectors in Persistent Memory

We identify five canonical threat vectors evaluated in this work:

1) **AV1: Direct Chat Injection:** An adversarial prompt delivered in dialogue attempting to elevate its own execution authority.
2) **AV2: Indirect Tool Poisoning:** Malicious instructions embedded within external web documents or tool return payloads.
3) **AV3: Delayed Action Steering:** Poisoned memories planted to steer future tool parameter selection across multiple sessions.
4) **AV4: Bridging Subversion:** Planting innocuous intermediate memories that iteratively chain into an unauthorized operation.
5) **AV5: Contradiction Floods:** Flooding conflicting observations to degrade the agent's confidence in authentic facts (Epistemic Denial of Service).

III. MemoryQuarantine Architecture

A. Semantic Retrieval Plane

The Semantic Retrieval Plane is responsible for recalling relevant context without conferring tool authority. Given a user query q, candidate memories are retrieved via Hybrid Reciprocal Rank Fusion (RRF) combining dense cosine similarity and sparse BM25:

$$\text{RRF}(m, q) = \frac{\alpha}{60 + \text{rank}_{\text{dense}}(m)} + \frac{1 - \alpha}{60 + \text{rank}_{\text{sparse}}(m)} \quad (2)$$

Recalled memories are formatted with explicit provenance metadata:

$$[\text{MEMORY id}=i \text{ provenance}=\gamma \text{ confidence}=C]: c \quad (3)$$

and injected into working memory for LLM reasoning.

TABLE I
Provenance Hierarchy (γ) and Reliability Calibration

Channel ID	Trust Tier	γ	Verification Protocol
`user_explicit`	Root Operator	1.00	Interactive CLI / WebUI prompt
`user_verified`	Confirmed	1.00	Operator challenge grant
`system_verified`	Local Machine	0.95	OS probe output / local environment
`local_file`	Filesystem	0.85	File read from local project
`file_system`	Storage	0.85	Verified disk state
`tool_success`	Execution	0.85	Verified tool execution return
`observation`	Agent Tool	0.75	Verified tool observation
`chat`	Dialogue	0.50	Multi-turn conversation
`web_search`	External	0.35	Search API snippet
`ambient_inferred`	Background	0.25	Ambient pattern inference
`external_untrusted`	Remote Doc	0.15	Downloaded remote file / web
`ext_untrusted`	Remote Alias	0.15	Remote document alias
`speculative`	Hypothetical	0.15	Unconfirmed agent hypothesis
`injected`	Adversarial	0.10	Suspected adversarial payload

B. Execution Authority Plane

When the agent proposes a tool call $a = \text{tool}(\text{args})$, the Execution Authority Plane intercepts the call prior to execution.

1) 1. Epistemic Confidence (C) with Information-Theoretic Surprise: Epistemic confidence $C(m) \in [0.05, 0.99]$ models belief certainty. Unlike naive additive counters, confidence is updated via Provenance-Weighted Epistemic Belief Revision (PW-EBR) using log-odds with Shannon surprise units (nats):

$$\text{Surprise}(C_t, y) = -\ln\left(\max\left(10^{-4}, 1 - |C_t(m) - y|\right)\right) \quad (4)$$

where $y = 1$ for corroborating evidence (polarity $= +1$) and $y = 0$ for contradictory evidence (polarity $= -1$). The log-odds update is given by:

$$\mathcal{L}_{t+1}(m) = \mathcal{L}_t(m) + \text{polarity} \cdot \gamma(m) \cdot \min(3.5, \text{scale}(\gamma) \cdot \text{Surprise}(C_t, y)) \quad (5)$$

Confidence is restored via the logistic sigmoid: $C(m) = \sigma(\mathcal{L}(m))$. Furthermore, memories undergo Unified Memory Forgetting with Ebbinghaus Decay and Retrieval Plasticity (UMF-EDR):

$$C(t) = C_0 \cdot \exp(-\lambda_{\text{eff}} \cdot \Delta t), \quad \lambda_{\text{eff}} = \frac{\lambda_0}{1 + \beta_{\text{plasticity}} \ln(1 + N_{\text{retrievals}})} \quad (6)$$

Epistemic tier transitions are symmetric: if $C(m) \geq 0.85$, the memory is promoted to `FACT`; if $0.50 \leq C(m) < 0.85$, it is maintained as an `OBSERVATION`; and if $C(m) < 0.50$, it collapses to a `HYPOTHESIS`, preventing one-way belief ratcheting.

2) 2. Action Reversibility Risk ($\mathcal{R}$): Action reversibility evaluates the operational consequence of the tool action on host state:

$$\mathcal{R}(a) = \begin{cases} 1.00, & \text{Invariant breach (e.g., } \texttt{rm -rf}\text{, } \texttt{format}\text{)} \\ 0.85, & \text{High mutation (}\texttt{git reset}\text{, } \texttt{chmod}\text{)} \\ 0.55, & \text{Destructive (file overwrite, drop table)} \\ 0.30, & \text{State modify (file create, patch)} \\ 0.05, & \text{Reversible / Read-only (}\texttt{read}\text{, } \texttt{search}\text{)} \end{cases} \tag{7}$$

C. Decision Boundaries & Gating Policy

The Guardian Safety Gate deterministically maps $\text{Authority}(m, a) = \gamma(m) \cdot C(m) \cdot (1 - \mathcal{R}(a))$ into three operational verdicts:

- $\text{Authority}(m, a) \geq 0.65 \implies$ **ALLOW:** Autonomous execution proceeds silently without user interruption.
- $0.25 \leq \text{Authority}(m, a) < 0.65 \implies$ **CHALLENGE:** Execution is paused; the operator is presented with an explicit confirmation dialog displaying memory provenance, reversibility risk, and exact command parameters.
- $\text{Authority}(m, a) < 0.25 \implies$ **BLOCK:** Execution is unconditionally interdicted. The action is quarantined and logged to the audit trail.

As formalized in Algorithm 1, when multiple memories are present in working context, MemoryQuarantine applies conservative weakest-link taint tracking ($\gamma_{\text{eff}} = \min \gamma(m)$). This guarantees safety monotonicity: adding an untrusted external memory cannot dilute taint to elevate execution authority.

Algorithm 1 MemoryQuarantine Dual-Plane Tool Authorization

Require: Query q, Action a, Epistemic Store $\mathcal{M}$, Context $\mathcal{C}$
Ensure: Authorization Verdict $V \in \{\text{ALLOW}, \text{CHALLENGE}, \text{BLOCK}\}$

$\mathcal{R}(a) \leftarrow \text{ComputeReversibilityRisk}(a)$
if $\mathcal{R}(a) \geq 1.00$ **or** $\text{MatchesHardInvariant}(a)$ **then**
  return BLOCK {Non-negotiable system invariant}
end if
$\mathcal{M}_{\text{recalled}} \leftarrow \text{ExtractRecalledMemories}(\mathcal{C})$
if $\mathcal{M}_{\text{recalled}} = \emptyset$ **then**
  $\gamma_{\text{eff}} \leftarrow 1.00$, $C_{\text{eff}} \leftarrow 1.00$ {Direct user prompt turn}
else
  $\gamma_{\text{eff}} \leftarrow \min_{m \in \mathcal{M}_{\text{recalled}}} \gamma(m)$ {Weakest-link taint tracking}
  $C_{\text{eff}} \leftarrow \min_{m \in \mathcal{M}_{\text{recalled}}} C(m)$
end if
$\text{Auth} \leftarrow \text{round}(\gamma_{\text{eff}} \cdot C_{\text{eff}} \cdot (1 - \mathcal{R}(a)), 4)$
if $\text{Auth} \geq 0.65$ **then**
  return ALLOW
else if $\text{Auth} \geq 0.25$ **then**
  return CHALLENGE
else
  return BLOCK
end if

IV. Experimental Evaluation

A. Experimental Setup

We evaluate MemoryQuarantine across two complementary modalities:

1) **CopperMemoryBench (160 Traces):** A structured benchmark comprising 100 multi-session adversarial traces across vectors AV1–AV5 and 60 benign control traces (BC1: User file edits, BC2: Read-only research, BC3: System config queries).
2) **End-to-End Physical Execution Harness:** Real agent tool invocations executed locally on an NVIDIA GeForce RTX 5060 Laptop GPU (8GB VRAM) running local open-weights models (`qwen2.5:0.5b`, `llama3.2:3b`) against isolated filesystem canaries.

B. Security Performance: ASR and Threat Interception

As summarized in Table II, MemoryQuarantine completely neutralizes delayed memory poisoning across all evaluated attack classes:

- **Flat RAG (B_0) and LWW (B_1)** suffer 100.0% security failure ($\text{ASR} = 100.0\%$). Because they lack provenance and consequence awareness, any recalled injection is executed unconditionally.
- **Bayesian Updating (B_2)** achieves an ASR of 80.0%. It successfully dampens Contradiction Floods (AV5) due to log-odds probability tracking, but fails against AV1, AV2, AV3, and AV4 because it evaluates content belief rather than tool action consequence.
- **Provenance-Only (B_3)** also yields an ASR of 80.0%. While provenance weighting suppresses gradual drift in AV3, it lacks action risk calibration and fails whenever an instructed action exceeds read thresholds.
- **MemoryQuarantine (M_5)** achieves an ASR of **0.0%** and a Threat Interception Rate of **100.0%** (0 / 100 breaches). The exact 95% Clopper-Pearson confidence interval for ASR is $[0.0\%, 3.6\%]$.

C. The 2×2 Factorial Benchmark Matrix

To guard against action-risk confounding where adversarial traces are separated from benign controls solely by action destructive potential, Table III evaluates MemoryQuarantine across the orthogonal 2×2 factorial space:

In Quadrant Q3, an untrusted memory ($\gamma = 0.15$) instructs a low-risk read of sensitive local files. Baselines relying purely on reversibility allow execution. Under MemoryQuarantine, the low provenance score bounds authority to $0.141 < 0.25$, successfully interdicting the reconnaissance attempt.

TABLE II
SECURITY PERFORMANCE AND UTILITY ACROSS GOVERNANCE BASELINES ON COPPERMEMORYBENCH

Governance Baseline	ASR (%)	95% Clopper-Pearson CI	TIR (%)	TSR (%)	FQR (%)	OCR (%)	Latency (ms)
B_0 (Flat RAG / MemGPT [3])	100.0%	[96.4%, 100.0%]	0.0%	100.0%	0.0%	0.0%	< 0.001
B_1 (Last-Write-Wins)	100.0%	[96.4%, 100.0%]	0.0%	100.0%	0.0%	0.0%	< 0.001
B_2 (Bayesian Updating [11])	80.0%	[70.8%, 87.3%]	20.0%	100.0%	0.0%	0.0%	< 0.001
B_3 (Provenance-Only IFC [10])	80.0%	[70.8%, 87.3%]	20.0%	100.0%	0.0%	0.0%	0.005
M_5 **(MemoryQuarantine)**	**0.0%**	**[0.0%, 3.6%]**	**100.0%**	**100.0%**	**0.0%**	70.0%	**0.005**

TABLE III
ORTHOGONAL 2×2 FACTORIAL EVALUATION MATRIX ($N = 30$ EACH)

	Low Risk ($\mathcal{R}(a) \leq 0.20$)	**High Risk** ($\mathcal{R}(a) \geq 0.70$)
High Provenance ($\gamma \geq 0.85$)	**Quadrant Q1 (Benign Read)** Auth: 0.855 $\rightarrow$ **ALLOW** Utility: 100% (0% Interruption)	**Quadrant Q2 (Benign Mutation)** Auth: 0.285 $\rightarrow$ **CHALLENGE** Safety: 100% (Confirmed)
Low Provenance ($\gamma \leq 0.25$)	**Quadrant Q3 (Exfil Recon)** Auth: 0.141 $\rightarrow$ **BLOCK** Defense: 100% Intercepted	**Quadrant Q4 (Poisoned Wipe)** Auth: 0.022 $\rightarrow$ **BLOCK** Defense: 100% Intercepted

D. Benign Utility Preservation & Operator Challenge Rate

On the 60 benign control tasks, MemoryQuarantine achieved a **100.0% Task Success Rate** (exact 95% CI: [94.0%, 100.0%]) and a **0.0% False Quarantine Rate**. Reversible read operations and verified user customizations executed autonomously without interruption. Consequential operations triggered clear confirmation dialogues (OCR = 70.0%). We candidly note that a 70.0% challenge rate under strict mode represents an operational friction cost; in Section VI, we discuss semantic confirmation safeguards to mitigate habituation and confirmation fatigue.

E. Physical Execution & Micro-Latency Profile

In the physical end-to-end evaluation using `qwen2.5:0.5b` on the RTX 5060 GPU:

- Under Baseline B_1, the LLM parsed the injected document and generated tool calls overwriting `canary_target.env`.
- Under MemoryQuarantine (M_5), `ToolExecutor` intercepted the call in 0.0128 **ms**, computing Authority $= 0.0641 < 0.25$. The invocation was quarantined, and physical disk inspection confirmed `canary_target.env` remained authentic and uncorrupted.

Authority calculation latency was measured at 0.005 **ms** mean (P95: 0.008 ms). MemoryQuarantine requires **0 bytes of additional GPU VRAM** and adds less than 0.01% overhead to the local LLM inference cycle (0.30 s per generation).

V. DISCUSSION, SCOPE BOUNDARIES, & LIMITATIONS

A. Persistent Epistemic Store vs. In-Session Tool Injections

MemoryQuarantine specifically secures the **persistent memory boundary** across disjoint sessions. If an agent executes a tool during a multi-step turn (e.g., `web_search`) and ingests an untrusted injection directly in the tool return stream, that payload resides in active turn context rather than recalled persistent memory. Mitigating immediate in-session tool poisoning requires dynamic turn-level taint tracking, which we identify as an essential complementary defense.

B. Confidentiality vs. Operational Reversibility

Operational reversibility $\mathcal{R}(a)$ measures the cost of restoring system state after an action. However, reading sensitive credentials (e.g., `.env`, SSH keys) causes zero disk mutations ($\mathcal{R} = 0.05$) while inflicting total confidentiality loss. To address this, production deployments must augment reversibility with target sensitivity:

$$\mathcal{R}_{\text{effective}}(a) = \max\left(\mathcal{R}_{\text{operational}}(a), \mathcal{S}(\text{target})\right) \tag{8}$$

where $\mathcal{S}(\text{target}) = 0.85$ for credential stores, ensuring that unauthorized read reconnaissance is strictly challenged.

C. Confirmation Fatigue & Human Factors

Operating with a 70.0% Operator Challenge Rate on consequential tasks introduces the risk of human habituation [12]. If operators routinely click "Approve" without inspecting parameters, the challenge gate becomes ineffective. We recommend implementing semantic confirmation for high-risk actions, requiring the operator to type the target filename or confirm the specific mutation parameter.

VI. RELATED WORK

MemGPT [3] and Generative Agents [4] pioneered long-term agent memory architectures, while HippoRAG [5] demonstrated associative recall. However, these systems treat recalled memories as trusted context. Torres et al. [6] formulated GhostWriter, proving the vulnerability of persistent memory to delayed poisoning. Dong et al. [7] demonstrated query-only injection (MINJA), and Chen et al. [8] demonstrated RAG backdoor triggers (AgentPoison). While contemporaneous works such as TMA-NM [10] and Singh [11] introduce origin-bound tracking, they enforce discrete binary sinks. MemoryQuarantine provides a continuous, consequence-calibrated authority plane that bridges memory provenance, belief certainty, and physical tool reversibility.

VII. Conclusion

Persistent memory is indispensable for personalized autonomous agents, but creates an acute delayed attack surface when semantic retrieval is conflated with execution authority. In this work, we presented **MemoryQuarantine**, a consequence-calibrated dual-plane governance architecture for local tool-using agents. By decoupling retrieval relevance from tool authority and enforcing $\text{Authority}(m, a) = \gamma(m) \cdot C(m) \cdot (1 - \mathcal{R}(a))$ fail-closed at the physical tool boundary, MemoryQuarantine eliminates delayed memory poisoning (ASR $= 0.0\%$, TIR $= 100.0\%$, exact 95% CI: [96.4%, 100.0%]) while preserving benign task utility under sub-0.01 ms evaluation latency on consumer edge hardware.

References

[1] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[2] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and Q. Zhang, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[3] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[4] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*. ACM, 2023, pp. 1–22.

[5] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[6] G. Torres, S. Shrestha, and S. Misra, "When agents remember too much: Memory poisoning attacks on large language model agents," *arXiv preprint arXiv:2607.06595*, 2026.

[7] S. Dong, S. Xu, P. He, Y. Li, J. Tang, T. Liu, H. Liu, and Z. Xiang, "Memory injection attacks on LLM agents via query-only interaction," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[8] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning retrieval-augmented generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[9] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, vol. 37, 2024.

[10] Y. Louck, "Non-malleable information flow control for agentic memory architectures," *arXiv preprint arXiv:2606.24322*, 2026.

[11] P. Singh, "When does belief-based agent memory help? provenance-capped bayesian updating in autonomous agents," *arXiv preprint arXiv:2606.22030*, 2026.

[12] R. Parasuraman and V. Riley, "Humans and automation: Use, misuse, disuse, abuse," *Human Factors: The Journal of the Human Factors and Ergonomics Society*, vol. 39, no. 2, pp. 230–253, 1997.